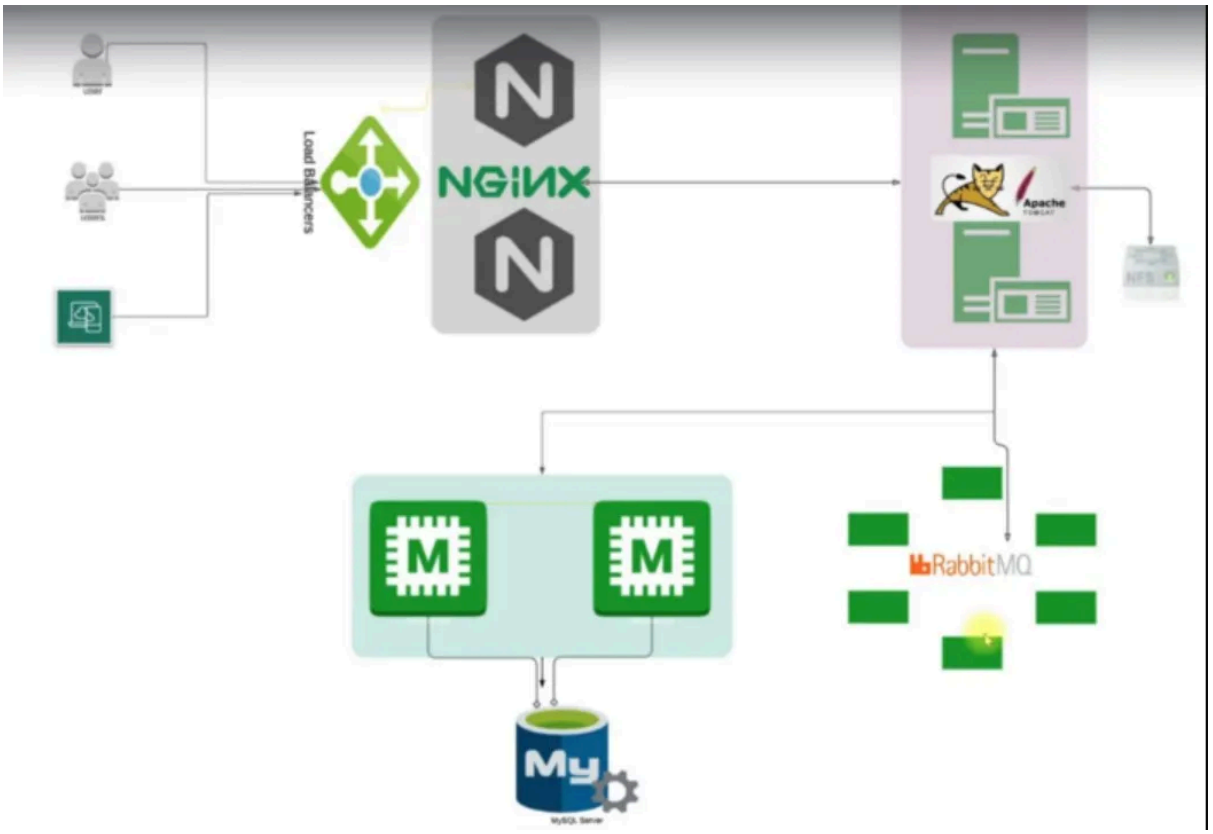


Rapport Complet : R&D DevOps, V-Profile Project & NFS Integration

📘 1. Introduction à la R&D en DevOps

🎯 Définition

La R&D DevOps est la discipline qui combine recherche, expérimentation et innovation dans les pratiques DevOps avant leur implémentation en production.

🔬 Activités Concrètes

- Benchmarking d'outils : Comparaison Jenkins vs GitLab CI, Ansible vs

Terraform

- Prototypage d'architectures : Tests de microservices vs monolithes
- Expérimentation de patterns : Blue-green deployment, canary releases
- Validation de technologies : Conteneurs, orchestration, monitoring
- Automatisation avancée : Self-healing systems, AIOps

📊 Processus Type

```
text

Idée → Recherche → Prototype → Test → Documentation → Implémentation
```

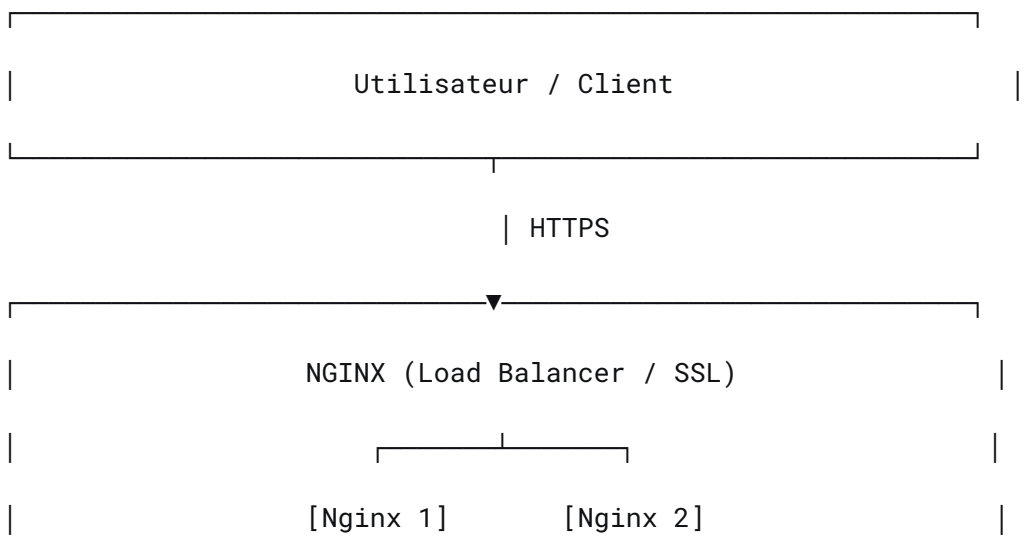
🏗️ 2. Projet V-Profile : Architecture Avancée

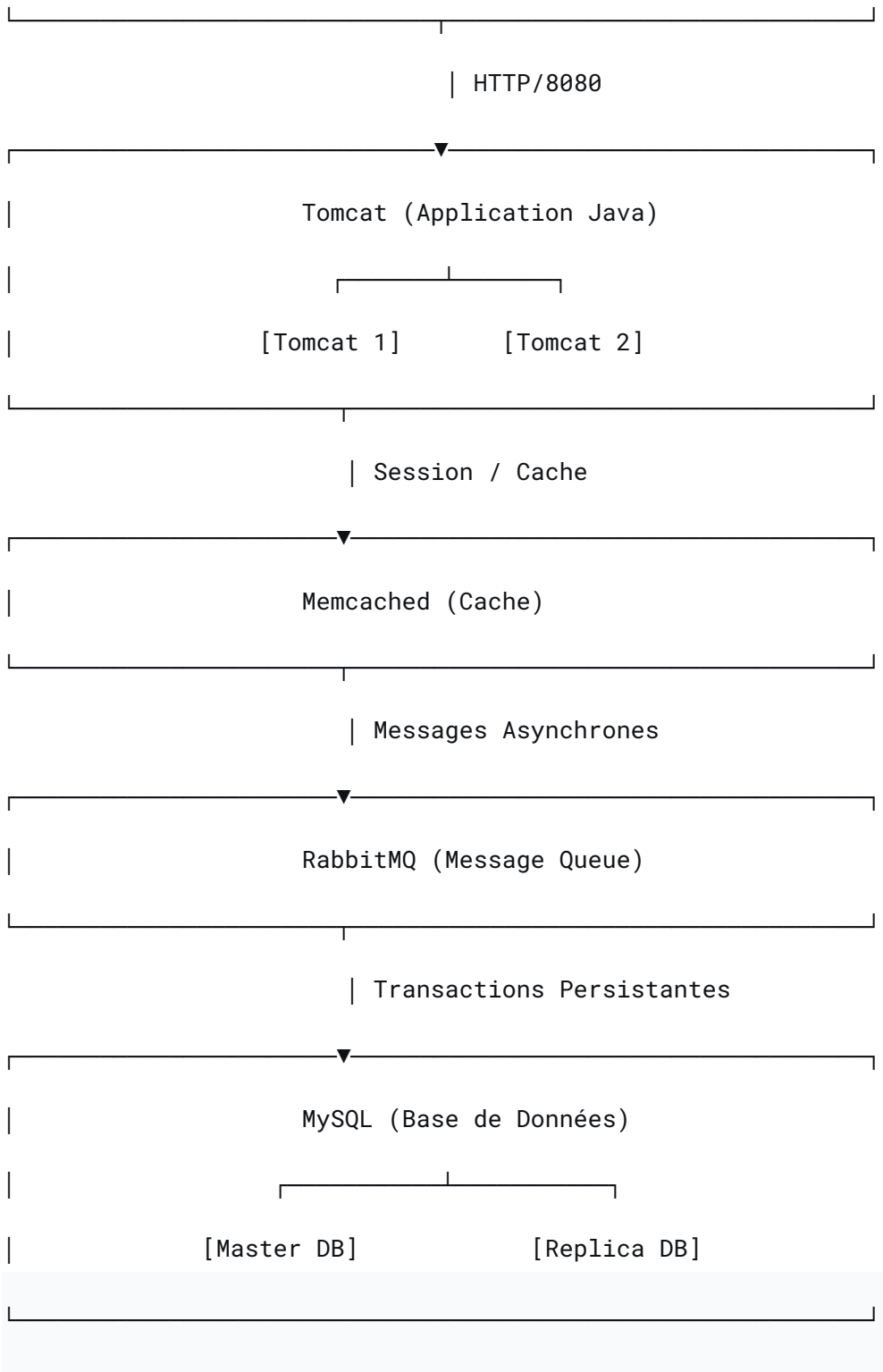
🎯 Objectif Principal

Application web Java de gestion de profils utilisateurs servant de laboratoire DevOps complet.

🧱 Architecture Multi-Tiers Détaillée

```
text
```





🛠️ Stack Technique Complète

Application Layer

- Langage : Java 11/17 + Spring Boot
- Build Tool : Maven / Gradle

- Serveur : Apache Tomcat 9+
- Frontend : HTML5, CSS3, JavaScript
- API : RESTful APIs

Infrastructure Layer

- Virtualisation : Vagrant + VirtualBox
- OS : Ubuntu 20.04 LTS / CentOS 8
- Cloud : AWS (EC2, VPC, ELB, RDS, EKS)

DevOps Toolchain

```
text

Git/GitHub → Jenkins → Ansible → Docker → Kubernetes → Monitoring
```

🔄 3. Phases de Développement & Automatisation

Phase 1 : Développement Manuel

```
bash

# Setup manuel

vagrant up

ssh vagrant@vm

sudo apt install nginx tomcat9 mysql-server memcached rabbitmq-server

# Configuration manuelle de chaque service
```

Phase 2 : Infrastructure as Code

```
yaml

# Vagrantfile

Vagrant.configure("2") do |config|
```

```
  config.vm.define "web" do |web|
    web.vm.box = "ubuntu/focal64"
    web.vm.network "private_network", ip: "192.168.56.10"
    web.vm.provision "ansible" do |ansible|
      ansible.playbook = "provisioning/playbook.yml"
    end
  end
end
```

Phase 3 : Pipeline CI/CD Jenkins

```groovy
pipeline {
    agent any
    stages {
        stage('Checkout') {
            steps { git 'https://github.com/vprofile-project' }
        }
        stage('Build') {
            steps { sh 'mvn clean package' }
        }
        stage('Test') {
            steps { sh 'mvn test' }
        }
        stage('Deploy to Dev') {
            steps { sh 'ansible-playbook deploy-dev.yml' }
        }
```

```
        stage('Deploy to Prod') {
            when { branch 'main' }
            steps { sh 'ansible-playbook deploy-prod.yml' }
        }
    }
}
```

Phase 4 : Conteneurisation Docker

```
dockerfile
# Dockerfile pour Tomcat
FROM tomcat:9-jdk11
COPY target/vprofile-app.war /usr/local/tomcat/webapps/
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

Phase 5 : Orchestration Kubernetes

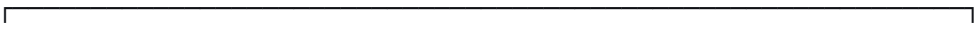
```
yaml
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vprofile-app
spec:
  replicas: 3
  selector:
    matchLabels:
```

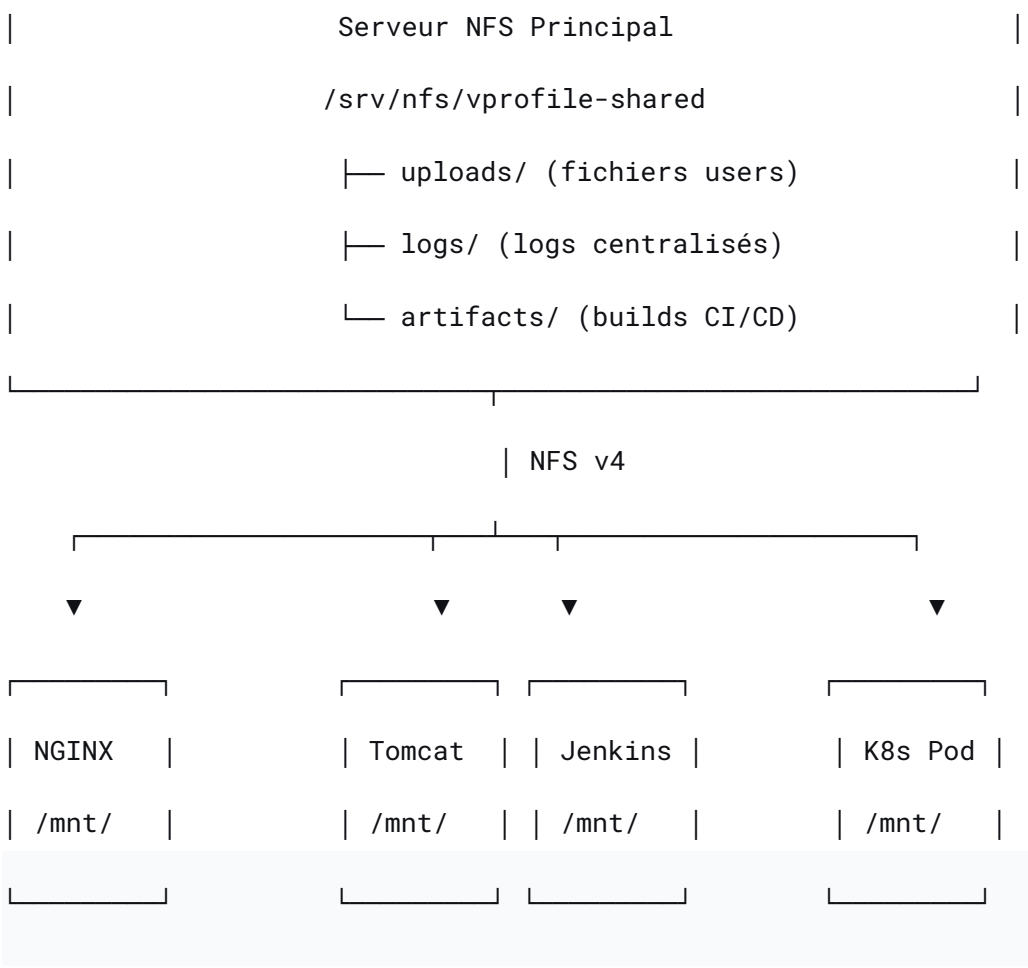
```
      app: vprofile
  template:
    metadata:
      labels:
        app: vprofile
    spec:
      containers:
      - name: vprofile
        image: vprofile-app:latest
        ports:
        - containerPort: 8080
        volumeMounts:
        - name: shared-storage
          mountPath: /app/data
      volumes:
      - name: shared-storage
        nfs:
          server: nfs-server-ip
          path: /srv/nfs/shared
```

📁 4. Intégration NFS dans l'Architecture V-Profile

🎯 Rôle du NFS dans l'Architecture

```
text
```





🔧 Configuration NFS Optimisée pour V-Profile

Serveur NFS (Dédié ou existant)

```
bash
# Installation
sudo apt install nfs-kernel-server nfs-common


# Création des répertoires partagés
sudo mkdir -p /srv/nfs/vprofile/{uploads,logs,artifacts,config}
sudo chmod -R 777 /srv/nfs/vprofile/  # Pour démo seulement


# Configuration /etc/exports
```

```
/srv/nfs/vprofile/uploads
192.168.56.0/24(rw,sync,no_subtree_check,all_squash,anonuid=1000,anongid=10
00)

/srv/nfs/vprofile/logs      192.168.56.0/24(rw,sync,no_subtree_check)

/srv/nfs/vprofile/artifacts 192.168.56.0/24(rw,sync,no_subtree_check)

/srv/nfs/vprofile/config    192.168.56.0/24(ro,sync,no_subtree_check)  #
Lecture seule pour config


# Appliquer

sudo exportfs -arv

sudo systemctl restart nfs-kernel-server
```

Client NFS (Chaque VM/Service)

```
bash
# Montage pour Tomcat

sudo mkdir -p /mnt/vprofile/{uploads,logs}

sudo mount 192.168.56.100:/srv/nfs/vprofile/uploads /mnt/vprofile/uploads

sudo mount 192.168.56.100:/srv/nfs/vprofile/logs /mnt/vprofile/logs


# Montage automatisé via /etc/fstab

192.168.56.100:/srv/nfs/vprofile/uploads /mnt/vprofile/uploads nfs defaults
0 0
```

🚀 Cas d'Usage Concrets dans V-Profile

1. Stockage Partagé pour Uploads Utilisateurs

```
java
// Configuration Spring pour utiliser NFS
```

```
@Configuration
public class StorageConfig {

    @Bean
    public Resource uploadDir() {
        return new FileSystemResource("/mnt/vprofile/uploads");
    }

    @Bean
    public CommonsMultipartResolver multipartResolver() {
        CommonsMultipartResolver resolver = new CommonsMultipartResolver();
        resolver.setMaxUploadSize(10485760); // 10MB
        resolver.setUploadTempDir(uploadDir());
        return resolver;
    }
}
```

2. Centralisation des Logs

```
xml
<!-- logback.xml pour Tomcat -->
<configuration>
    <appender name="FILE" class="ch.qos.logback.core.FileAppender">
        <file>/mnt/vprofile/logs/vprofile-app.log</file>
        <encoder>
            <pattern>%d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger{36}
- %msg%n</pattern>
        </encoder>
```

```
    </appender>

    <root level="INFO">
        <appender-ref ref="FILE" />
    </root>
</configuration>
```

3. Artifacts CI/CD Partagés

```
groovy
// Pipeline Jenkins utilisant NFS
pipeline {
    environment {
        ARTIFACT_DIR = '/mnt/vprofile/artifacts'
    }

    stages {
        stage('Build') {
            steps {
                sh 'mvn clean package'
                sh "cp target/*.war
${ARTIFACT_DIR}/vprofile-\${BUILD_NUMBER}.war"
            }
        }
    }
}
```

4. Kubernetes Persistent Volume avec NFS

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: vprofile-nfs-pv
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteMany
  nfs:
    server: 192.168.56.100
    path: "/srv/nfs/vprofile"
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: vprofile-nfs-pvc
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 5Gi
```

🔄 5. Workflow DevOps Complet avec NFS

📋 Scénario : Déploiement d'une Nouvelle Version

```
text
```

```
Étape 1 : Développeur push sur Git
          |
          ▼
Étape 2 : Jenkins détecte le changement
          |
          ▼
Étape 3 : Build + Tests automatiques
          |
          ▼
Étape 4 : Artifact stocké sur NFS (/artifacts)
          |
          ▼
Étape 5 : Déploiement sur Tomcat via Ansible
          |
          ▼
Étape 6 : Application utilise NFS (/uploads, /logs)
          |
          ▼
Étape 7 : Monitoring via logs centralisés
          |
          ▼
```

```
Étape 8 : Rollback si nécessaire (artifacts sur NFS)
```

🛡️ Sécurité NFS pour Environnement de Production

```bash
# Meilleures pratiques de sécurité

# 1. Restreindre les adresses IP

/srv/nfs/vprofile 192.168.56.10(rw) 192.168.56.11(rw)


# 2. Utiliser Kerberos pour l'authentification

/secured_nfs gss/krb5i(rw,sync,no_subtree_check)


# 3. Exports en lecture seule quand possible

/srv/nfs/vprofile/config 192.168.56.0/24(ro)


# 4. Firewall approprié

sudo ufw allow from 192.168.56.0/24 to any port nfs
```

📊 6. Métriques et Monitoring

📈 Métriques Clés à Surveiller

```yaml
Infrastructure:

  - Utilisation NFS: df -h /mnt/vprofile

  - IOPS NFS: iostat -x 1

  - Latence réseau: ping nfs-server
```

```
Application:
  - Taux de succès upload: logs NFS
  - Taille moyenne fichiers: du -sh /mnt/vprofile/uploads
  - Erreurs de montage: /var/log/syslog

Business:
  - Nombre fichiers uploadés/jour
  - Espace stockage consommé
  - Temps d'accès moyen
```

🚨 Alertes Recommandées

```
bash
# Script de monitoring NFS
#!/bin/bash
NFS_MOUNT="/mnt/vprofile"
THRESHOLD=85

USAGE=$(df -h $NFS_MOUNT | awk 'NR==2 {print $5}' | sed 's/%//')

if [ $USAGE -gt $THRESHOLD ]; then
    echo "ALERTE: NFS usage à $USAGE%" | mail -s "NFS Alert"
admin@vprofile.com
fi
```

🏆 7. Avantages de cette Architecture Combinée

✅ Bénéfices pour V-Profile

1. Centralisation : Tous les services accèdent aux mêmes données
2. Persistance : Données survivent aux redémarrages de containers
3. Scalabilité : Facile d'ajouter de nouveaux services
4. Backup simplifié : Un seul point à sauvegarder
5. Debugging facilité : Logs centralisés

🔄 Améliorations Possibles

1. HA NFS : Cluster NFS avec DRBD + Pacemaker
2. Caching : Utiliser cache NFS (fscache)
3. Migration Cloud : Remplacer NFS par EFS (AWS) ou Filestore (GCP)
4. Sécurité : Chiffrement des données au repos

📚 8. Conclusion et Next Steps

🎯 Acquis de l'Exercice

- ✅ Architecture multi-tiers réaliste
- ✅ Intégration complète DevOps
- ✅ Stockage partagé avec NFS
- ✅ Pipeline CI/CD opérationnel
- ✅ Préparation pour le cloud

🚀 Prochaines Étapes

1. Migrer vers Kubernetes : Déployer tous les services sur K8s
2. Ajouter un CDN : Pour les uploads utilisateurs
3. Implémenter du monitoring : Prometheus + Grafana
4. Automatiser les backups : NFS + bases de données

5. Tests de charge : Valider la performance NFS

📞 Support et Ressources

- Documentation NFS : `man nfs`
- Monitoring : Nagios/Zabbix pour NFS
- Backup : Rsync + cron vers S3
- Sécurité : Audit NFS avec `nfs4audit`